ALL THINGS AI

YOUR FIELD GUIDE

# Vibe Coding for Non-Technical Professionals

Your first website. Your first game. Your next idea.

01 UNDERSTAND

02 MAKE A WEBSITE

03 MAKE A GAME



A website. A game. A repeatable way to turn an idea into working software.

No programming experience required. Bring your favorite chatbot and learn to describe, build, check, save, and publish.

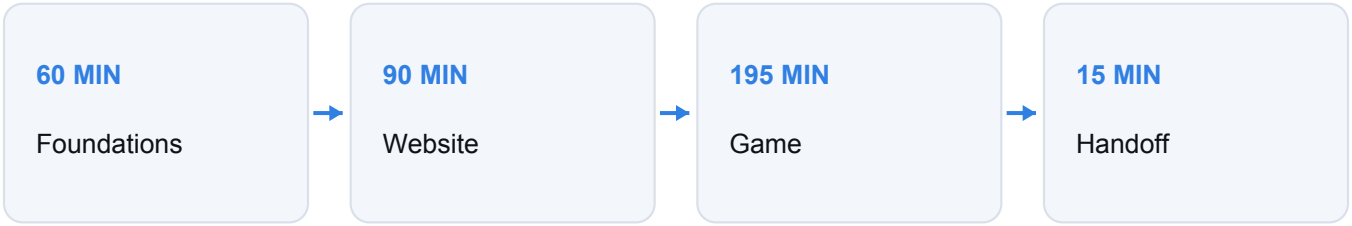
KEEP IN MIND

Six teaching hours, plus lunch and breaks.
Mark Hinkle | All Things AI
Participant edition | October 2026

01 / YOUR ROUTE

# Two projects. One useful process.

Use this guide as a map. The repository holds the detailed instructions and recovery steps.




A small checked result becomes the starting point for the next step.

01 FOUNDATIONS - 60 minutes. Understand the software lifecycle, then prepare GitHub, Vercel, and your chatbot.

02 WEBSITE - 90 minutes. Define one page, build it, test its main action, and publish it.

03 GAME - 195 minutes. Choose a pattern, write rules, build a playable loop, improve it, playtest, and publish.

04 HANDOFF - 15 minutes. Demonstrate both projects and save your next action.

**KEEP IN MIND**

Go at your own pace outside class. A checkpoint is a visible result, not a number of prompts sent.

Quick page finder: AI handoff 3; lifecycle 4; tools 5; setup 6; website 7-9; game 10-12; troubleshooting 13; agents 14-15; resume 16; checklist 17; resources 18.

02 / HAND THE COURSE TO YOUR AI

# Start with the right context.

A chatbot can help only with the material and tools it can actually access.

### COURSE MATERIAL

- AI-START.md
- Current lesson + skill
- Your question

### PROJECT CONTEXT

- Actual current files
- Version + observations
- Next small action

01 Open START-HERE.md. If your assistant can read the authorized repository, direct it to AI-START.md.

02 If it cannot read the private link, download AI-COURSE-PACK.md and attach it. If file upload fails, paste AI-START.md and the current lesson.

03 Ask it to identify the files it received, both projects, and your next checkpoint. A vague summary is not proof of access.

**TRY THIS PROMPT**

Be my patient course guide. Read AI-START.md and tell me which files you actually received. I am starting fresh / continuing / stuck. Give me one next action and tell me what success will look like.

**KEEP IN MIND**

The course repo is private. Your instructor must grant access or supply a copy. Never share a password or access token to make a link work. ZIP uploads are not supported by every chatbot.

03 / THE FIRST 30 MINUTES

# How software gets made.

The software development lifecycle is a set of questions you revisit as you learn.




Discover the user and purpose. Define the smallest useful result. Design the experience and plan small steps. Build one increment. Test the actual behavior. Release the checked version. Maintain it as needs change.

## Make each stage concrete

Website: a visitor should find event details and open a directions link. Game: a player should answer five questions, see an accurate score, and play again. Those promises become tests.

**TRY THIS PROMPT**

Explain our current stage using my project. What decision do I need to make, what file should we produce, and what can a person do to check the result?

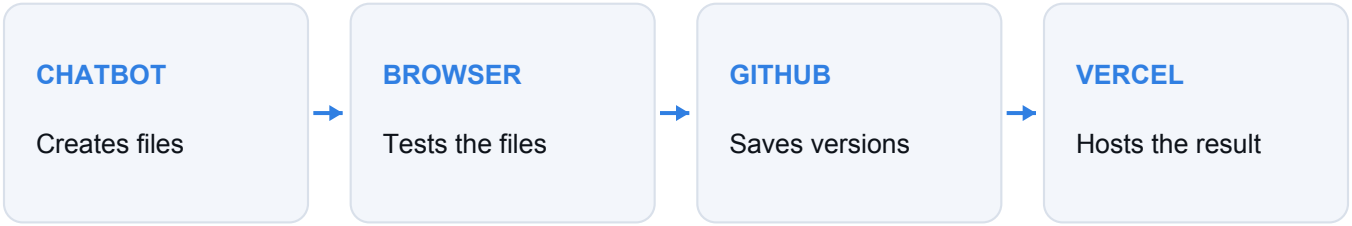
**KEEP IN MIND**

You can return to an earlier stage. A confusing result may need a better rule or clearer wording, not more code.

04 / THE TOOL MAP

# Know what each tool does.

The chatbot helps create files. Your browser shows what those files actually do.




A small checked result becomes the starting point for the next step.

HTML gives a page structure. CSS controls its appearance. JavaScript supplies behavior, such as answering a quiz or restarting a game. Both core projects can run as plain static files.

01 CHATBOT: creates or changes project files. Its response is a proposal until checked.

02 GITHUB: stores your files and their saved history. A commit is a named checkpoint; a branch is a separate line of changes.

03 VERCEL: hosts a deployed version at a URL. Your browser opens that URL; the local copy may differ.

**KEEP IN MIND**

A repository is not the same thing as a live website. A private repository also does not prove that its deployed website is private.

A pull request proposes a branch change for review. A preview deployment helps you inspect a change before a later production release.

05 / GET READY

# Choose the route that works.

Start with complete runnable files. Add connected tools only when they help.

**my-first-site**

- index.html
- project-brief.md
- project-state.md

**my-first-game**

- index.html
- game-rules.md
- project-state.md

Keep both projects separate.

01 Create or sign in to GitHub and Vercel using setup-guide.md. Keep your account recovery details private.

02 Choose your chatbot. Ask it for a downloadable index.html, or complete plain-text contents you can save.

03 Create a website folder and learner-owned repository. Keep the game in a second folder and repository later.

04 Open the exported file in your browser. Confirm the filename ends in .html, not .html.txt, and that it is plain text.

**TRY THIS PROMPT**

Confirm whether you can return complete files, edit my project, run checks, and access GitHub or Vercel. Mark each verified, unavailable, or not tried. Use the portable route for anything you cannot do here.

**KEEP IN MIND**

No API key, database, or paid integration is needed for the core projects. Confirm hosting suitability before class: Vercel Hobby is for noncommercial personal use. Native skills and multiple agents are optional.

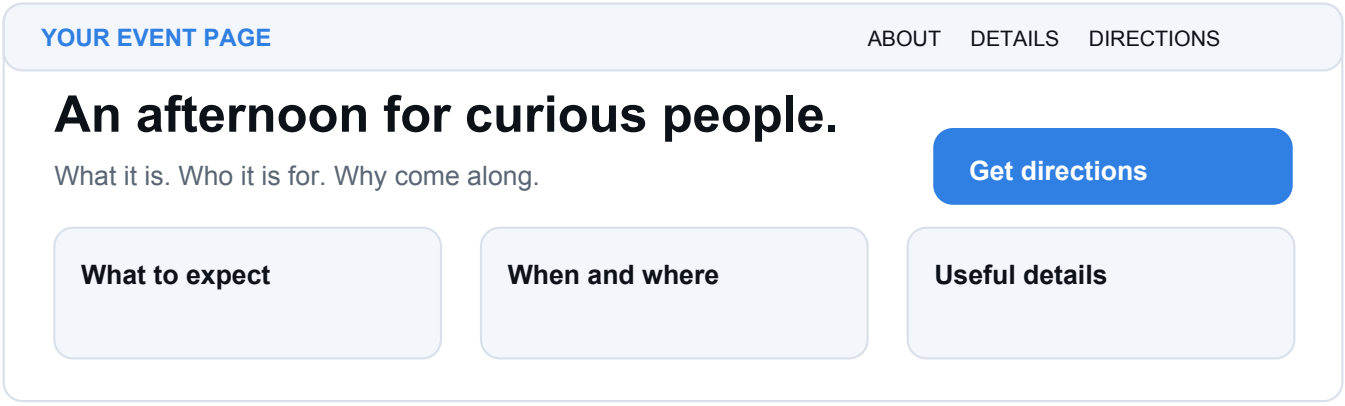


Open the detailed lesson: setup-guide.md

06 / WEBSITE BRIEF

# Give the page one clear job.

Start with a small, specific promise to one kind of visitor.




Choose a starter or make your own: vanilla, two All Things AI fan sites, dream restaurant, dream product, dream business, or pet. Download the course and open website-starters/index.html. Copy the chosen folder into your own project.

01 Write the audience and desired outcome in one sentence.

02 Scope WHAT: one page, three sections, one useful action. Put extra features in a later list.

03 Plan HOW: three small changes, each with a visible check. Check purpose, content, action, navigation, and phone layout.

**TRY THIS PROMPT**

Use vc-project-brief, vc-project-scope, and vc-project-plan. Help me define who and why, what belongs now, and how to build in three increments. Include the exact destination or behavior of the main action.

**KEEP IN MIND**

Save your brief, scope, and plan. Use content and images you have permission to use. Fictional examples should be visibly fictional; do not invent testimonials or working services.



Open the detailed lesson: docs/scope-and-plan.md

07 / WEBSITE BUILD

# Build small. Look. Then improve.

A specific change is easier to check than an instruction to make everything better.

### VAGUE

Make it better.

No clear scope.

No completion check.

### SPECIFIC

Make the button easy to tap.

Keep the link destination.

Check it on a phone.

01 Ask for the smallest complete page. A single index.html can contain its styles and behavior.

02 Save and open the actual exported file. If separate images or scripts are supplied, keep their paths intact.

03 Try the main action. Save a working copy before asking for one improvement.

04 Repeat the same check after the change and inspect anything that should have stayed the same.

**TRY THIS PROMPT**

Build the first increment from my brief. Return the complete runnable file. Then change only the heading and button placement for phone readers. Preserve my content, destinations, and working links.

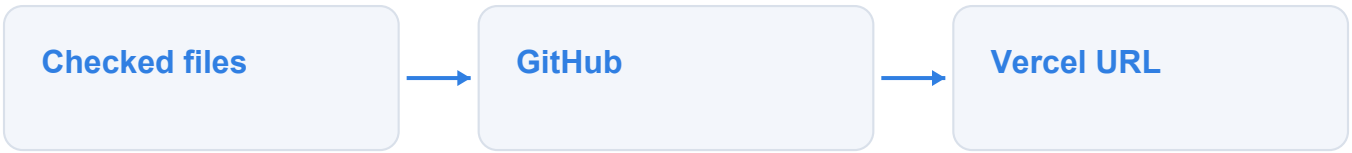
**KEEP IN MIND**

An image of a page is a design reference. It is not a runnable website. If the assistant only gives a screenshot, ask for the complete project files.

08 / WEBSITE RELEASE

# Check before you share.

Publishing and verification are two separate steps.




**FIRST DEPLOYMENT**

Production: confirm before sharing.

**LATER CHANGES**

Branch > preview > review > release

01 Have another person explain the page and follow its main action. Try all links, keyboard navigation, and a narrow screen. Record real observations.

02 Upload the checked files, not the ZIP, to your own website repository. Confirm index.html is at the selected project root.

03 Import that repository into the correct Vercel account/project. For plain static files, follow the Other / empty build command setup in setup-guide.md.

04 Confirm the intended publication, deploy, and repeat the main checks at the actual URL. Record that URL and the released version.

**KEEP IN MIND**

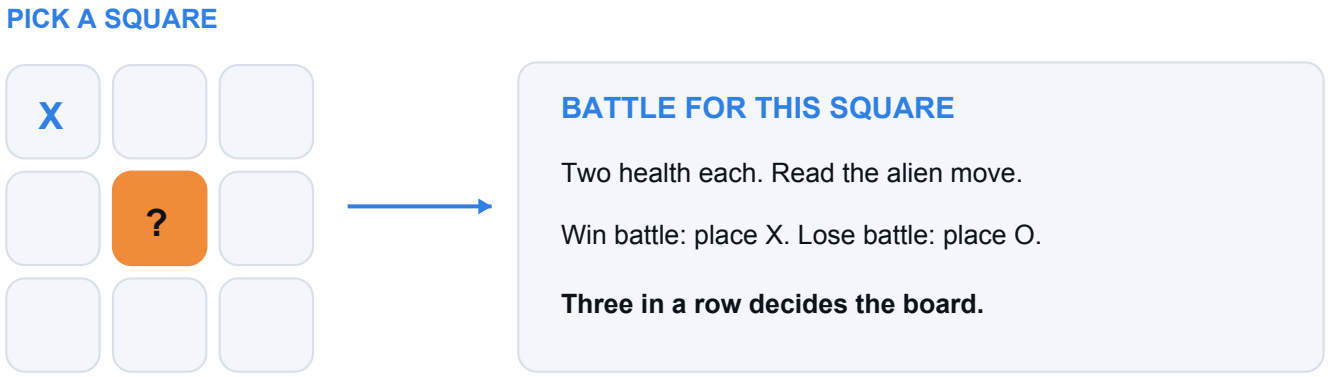
The first Vercel deployment is production. Later updates to the configured production branch can publish automatically. Use branches and previews for later changes.

If deployment fails, keep your working files and record the exact error. A green build indicator is not evidence that the main button works.

09 / GAME BRIEF

# Battle an alien. Claim a square.

Use a familiar pattern, then make the theme your own.




Pick one: Alien Tic-Tac-Toe (featured), a five-question quiz (easiest), or a one-button space dodger (action). Mark proposed the alien game; the alternatives draw on Arcade patterns.

## Alien Tic-Tac-Toe: a solo variant

Choose an empty square, then battle an alien. Win to claim X; lose and the alien claims O. Three Xs wins; three Os loses; a full board with no line draws. There is no extra opponent move. Build the board and short turn-based battle separately, then connect them.

**TRY THIS PROMPT**

Use vc-game-brief and vc-game-rules. Write the player goal, main action, keyboard and touch controls, score rule, round ending, and restart behavior. Give me five observable checks.

**KEEP IN MIND**

Keep the first version small. Accounts, multiplayer, global rankings, daily locks, and remote score services can wait. Educational facts need reliable sources; a game can simply be fun.

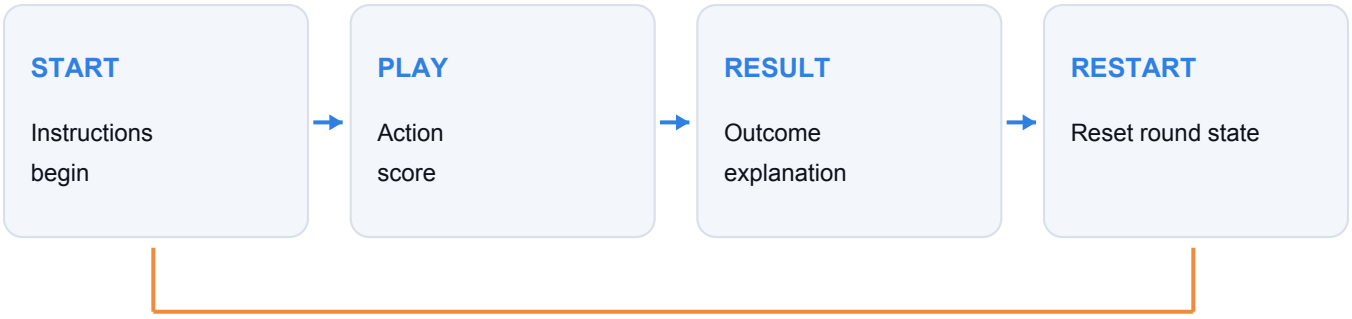


Open the detailed lesson: docs/game-options.md

10 / GAME STATE

# Make the whole loop work.

State is what the game remembers: where you are, the score, and whether the round is over.




Play again returns to a fresh round.

01 START: show short instructions and one clear way to begin.

02 PLAY: accept the intended input and update progress or score once.

03 RESULT: end the round, explain the outcome, and stop inappropriate input.

04 RESTART: reset the agreed state and begin a fresh round without duplicated timers or actions.

**TRY THIS PROMPT**

Build the complete start, play, result, and restart loop with plain visuals first. Keep rules separate from drawing where practical. Explain how I can test one round and three restarts.

**KEEP IN MIND**

Try rapid double-clicks or taps where relevant. A quiz should not award points twice; an animated game should not run faster after every restart. Make controls match the instructions.

11 / GAME PLAYTEST

# Watch a new player.

Two minutes without coaching can reveal what your own testing misses.

| | |
|---|---|
| PASSED | Observed the expected result. |
| FAILED | Tried it; the result was wrong. |
| NOT TESTED | No observation yet. |

01 Give the current version to a partner. Let them start, play, finish, and restart without hints.

02 Record hesitation, control errors, unfair outcomes, and scoring surprises. Ask whether they would replay and why.

03 Fix the biggest issue. Tune only one setting, such as speed or time, then repeat the same test.

04 Publish to a separate game repository and Vercel project. Repeat a round and restart at the deployed URL.

**TRY THIS PROMPT**

Review this exact version against game-rules.md. Separate observed failures from suspicions. Help me repair the biggest problem and retest it. Do not mark a phone test Passed if we only resized a desktop browser.

**KEEP IN MIND**

Use Passed, Failed, and Not tested. If you include local best scores, test refresh and unavailable storage. Sound is optional; gameplay should remain understandable without it.



Open the detailed lesson: docs/game-options.md

12 / GET UNSTUCK

# Diagnose one thing at a time.

Keep the last working version. Avoid turning a small failure into a full rewrite.

| | |
|---|---|
| 1 CAPTURE | Steps, expected result, actual result |
| 2 LOCATE | Right file, project, version, and URL |
| 3 REPAIR | One reversible change; repeat the check |
| 4 HAND OFF | Two failed repairs? Share the evidence |

01 NAME THE SYMPTOM: exact click or input, expected result, actual result, error text, file or URL, and browser/device.

02 CHECK THE CONTEXT: correct project, latest file, filename, folder path, branch, and deployment.

03 TRY ONE REPAIR: make the smallest reversible change and repeat the same check.

04 ESCALATE WITH EVIDENCE: after two unsuccessful focused repairs, summarize what happened and ask for help.

**TRY THIS PROMPT**

I expected [result] after [steps]. I observed [result/error] in [file/URL]. The last change was [change]. Help me test one likely cause. Preserve the working parts and say how we will know the fix worked.

**KEEP IN MIND**

Use docs/troubleshooting.md for symptom-by-symptom recovery, including blank pages, .html.txt, missing files, stale deployments, broken controls, agent conflicts, and lost chat context.

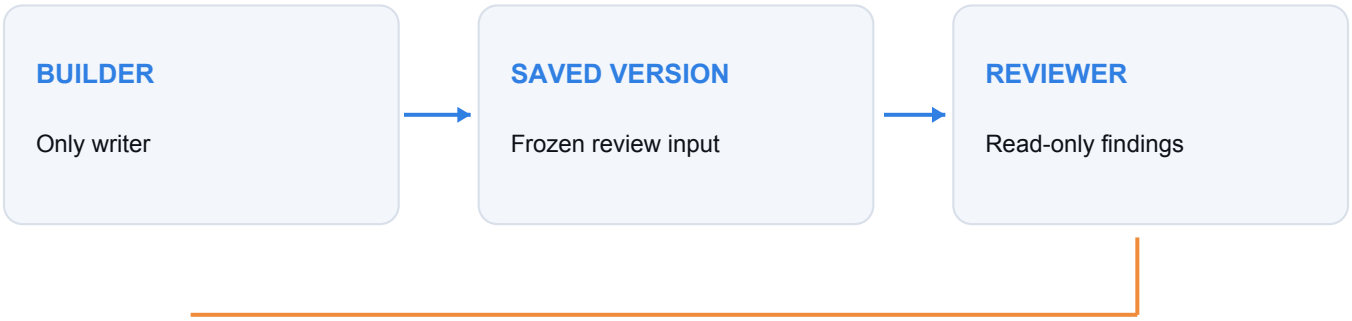


Open the detailed lesson: docs/troubleshooting.md

13 / OPTIONAL MULTIPLE AGENTS

# Give every helper a clear job.

Start with one builder and one reviewer. More agents are useful only when the work can be separated.




01 TWO CHATS: let one assistant build. Give another chat the same brief and a frozen copy to review read-only. Send useful findings back to the builder.

02 NATIVE SUBAGENTS: where supported, ask for one bounded helper, such as drafting acceptance checks. Keep one writer on the application files.

03 ISOLATED WORKTREES: advanced coding work can use separate folders and branches. A coordinator integrates the results and repeats the tests.

**TRY THIS PROMPT**

Use one helper to review this exact saved version read-only. Give it the brief and test criteria. Do not let it edit or publish. Return concrete issues with reproduction steps, then propose a small fix.

**KEEP IN MIND**

Different role names are not independent reviewers. Separate agents can still share a model or provider. A second provider is optional and requires a deliberate data-sharing choice. Extra agents consume extra usage; keep this outside the core timebox if it slows you down.

14 / YOUR PROJECT INSTRUCTIONS

# Give your AI a working agreement.

AGENTS.md is a short instruction file for your own project, not a magic installation command.

## AGENTS.md

| | |
|---|---|
| Project + audience | What we are making |
| Scope + files | What can change |
| Checks + evidence | How we know it works |
| State + next action | How another chat resumes |

01 Copy templates/AGENTS.md into your learner project as AGENTS.md. Fill in the project identity, intended user, and scope.

02 Add your expected behavior, how to open the project, how to test it, and where to record progress.

03 Ask your assistant to read and summarize the rules before making changes. In ordinary chat, paste or attach the file.

04 Use templates/CLAUDE.md only as described in the optional agent guide. Product discovery differs by client and version.

**TRY THIS PROMPT**

Read AGENTS.md and my current files. Tell me which project you are working in, the next allowed change, how you will test it, and what would require my decision. Do not claim you read instructions you could not access.

**KEEP IN MIND**

Use the sample inside templates/. The course root AGENTS.md routes assistants through the course; it is not the learner-project template. Instruction files guide behavior but do not replace platform permissions or tests.



Open the detailed lesson: templates/AGENTS.md

15 / PAUSE AND RESUME

# Save context outside the chat.

Your files and a short state record make it easier to switch assistants or continue tomorrow.

### COURSE MATERIAL

- AI-START.md
- Current lesson + skill
- Your question

### PROJECT CONTEXT

- Actual current files
- Version + observations
- Next small action

01 Save the latest working project files and name the checkpoint or commit.

02 Update project-state.md: goal, scope, folder/repository, current stage, tested behavior, unresolved issue, and one next action.

03 Give the next assistant both the state record and the actual files. Include the relevant course lesson or skill.

04 Ask it to identify the version and missing evidence before continuing. Keep publication permission tied to the intended destination.

**TRY THIS PROMPT**

Read my project-state.md and attached files. Summarize what works, what failed or was not tested, and the next action. Ask for missing files. Continue from the saved state without rebuilding everything.

**KEEP IN MIND**

Keep the website and game state records separate. Do not rely on an assistant remembering another chat or accessing a private conversation link.



Open the detailed lesson: templates/project-state.md

16 / FINISH WITH EVIDENCE

# What does done look like?

Show the result, identify the version, and state the limits.

| | |
|---|---|
| **PASSED** | Observed the expected result. |
| **FAILED** | Tried it; the result was wrong. |
| **NOT TESTED** | No observation yet. |

### Website

A visitor understands the purpose, reads accurate content, follows the primary action, uses navigation, and can read the page on a phone-sized screen. The published URL has been checked, or the deployment blocker is recorded.

### Game

A new player understands the instructions, uses the supported controls, receives correct scoring and a clear result, and can restart repeatedly. The deployed version has been played, or working files and the blocker are retained.

### Your handoff

Name the owner, files, repository, version, URL, test report, known limits, and next improvement. Demonstrate the website action and a full game round. Explain one assumption you corrected using evidence.

**KEEP IN MIND**

Published and checked; published but verification incomplete; or working files with deployment blocked. Use the statement that matches your evidence.

# The right help, one click away.

All core instructions are readable as text, so your chatbot can use them alongside you.

01 START-HERE.md - access the course, choose a route, and give your AI the starting prompt.

02 AI-COURSE-PACK.md - a single text file containing the current learning instructions, templates, and skills.

03 docs/website-walkthrough.md and docs/game-options.md - website steps and three game choices.

04 docs/troubleshooting.md - symptom-led recovery. templates/help-request.md - an evidence-based request for help.

05 docs/multiple-agents.md and templates/AGENTS.md - optional collaboration and project instructions.

06 handouts/student-workbook.md and templates/project-state.md - records you can reuse in both projects.

Proud of your game? Use docs/arcade-submission.md to prepare a review packet for Mark and All Things AI. Selected games may join the Arcade after later integration.

### Share your experience. Get a skill endorsement.

Add vibe coding to your LinkedIn profile, post an honest account of the class, and send Mark your profile and post links. Connect with him if needed. Mark will endorse the skill you practiced. Participation is optional; see docs/linkedin-wrap-up.md for the post example and details.

### Sources and edition notes

The setup guide cites official GitHub, Vercel, OpenAI, and Claude documentation. Product guidance was checked October 5, 2026; screens and account availability may change. The source map records the original website course, existing presentation, and Arcade research.

Brand palette and logo come from the existing All Things AI class presentation. Diagrams in this guide are conceptual illustrations, not screenshots of provider interfaces. The detailed Markdown is the living reference.

**KEEP IN MIND**

The guide is a learning aid. Website starters have local checks; game checkpoints and full classroom rehearsal remain preparation tasks. A documented workflow is not a claim that every provider or device has been exercised.